CAN A FLY'S GIVE-UP CIRCUIT LEARN TO GIVE UP? MULTI-AGENT REINFORCEMENT LEARNING IN THE WAR OF ATTRITION WITH CONNECTOME-WIRED POLICIES *

Suhani
IIT Gandhinagar
suhani.24110358@iitgn.ac.in

Mayank Choudhary
IIT Gandhinagar
mayank.choudhary@iitgn.ac.in

Krina Paghadar
IIT Gandhinagar
krina.paghadar@iitgn.ac.in

Nakshatra Lad
IIT Gandhinagar
nakshatra.lad@iitgn.ac.in

ABSTRACT

When two male fruit flies fight, one of them eventually gives up. The time at which it does so follows the exponential law that game theory predicts for the war of attrition, and recent work has traced this decision to a small circuit in the fly brain. The complete wiring diagram of the male fly nervous system now lets us copy that circuit neuron by neuron. We propose to turn the real give-up circuit into the policy network of a learning agent, let two such agents play the war of attrition against each other with reinforcement learning, and check whether the fly's wiring helps them find the equilibrium better or faster than networks of the same size with different wiring. This report sets up the game, explains how a circuit taken from the connectome becomes a trainable policy and how reinforcement learning updates it, and lists the comparisons, metrics and results we plan to show in the final submission.

Keywords multi-agent reinforcement learning · war of attrition · connectome · Drosophila · self-play · exploitability

1 Introduction

Picture two animals fighting over food, where neither can really hurt the other and the only question is who keeps going longer. Waiting costs energy, and whoever stays longest gets the food. In 1974 John Maynard Smith showed that in this game, the war of attrition, no fixed waiting time can be a good strategy [1]. If you always wait exactly t seconds, a rival who waits a little longer beats you every time. The only stable strategy is to wait a random amount of time, drawn from an exponential distribution whose average is the value of the food divided by the cost of waiting [1, 2]. For fifty years this was a prediction about behaviour, with nothing to say about how a brain could produce such a random but well-calibrated decision.

That is now changing. Ventimiglia et al. [3] found that fighting male fruit flies really do give up at exponentially distributed times, and that the rate shifts with hunger and with the presence of food, exactly as it should if flies value the prize more when they need it more. They also found the circuit behind the decision. PPL1 dopamine neurons carry the "this is going badly" signal, two mushroom-body output neurons (MBON-18 and MBON-19) combine it, and giving up shows up as a long-lasting shutdown of the Tk-GAL4^{FruM} aggression neurons. At the same time, complete wiring diagrams (connectomes) of the fly brain have become available: the hemibrain [4], FlyWire for the female brain [5], and now MaleCNS for the whole male nervous system [6]. So for the first time we can take the exact wiring of a circuit that carries out a game-theoretic strategy and ask what that wiring is good for.

Researchers have already used fly connectomes as neural network architectures. Lappalainen et al. [7] built networks wired like the fly visual system and found that the wiring alone predicts how real neurons respond. Jin et al. [8] turned

* *Project idea review*: CS329 Foundations of AI: Multi-Agent Systems, IIT Gandhinagar, 2026.

the whole fly brain into a graph neural network with fixed synapse weights and trained it with reinforcement learning to make a simulated fly body walk and fly [9]. Both treat the connectome as the controller of a single agent. Our setting is different because the war of attrition is a game between two agents. Its solution is a mixed equilibrium, not a single best action, and learning a mixed equilibrium through self-play is known to be tricky [10, 11].

Our question is this. If two learning agents play the war of attrition, and each agent's policy network is wired exactly like the real fly give-up circuit, do they learn the equilibrium better or faster than agents whose networks have the same size but different wiring?

Two things make this question clean to answer. First, the equilibrium of the game is known exactly, so we can measure how far a learned strategy is from it without any approximation, using the exploitability measure of Bichler et al. [12]. Second, we can build exact controls. A degree-preserving rewire [13] keeps how many connections each neuron has but shuffles who connects to whom, so any difference between the real circuit and its rewire comes from the specific wiring and nothing else. Either answer is useful. If the real wiring helps, the circuit may be shaped to learn this particular game well. If it does not, the circuit's role likely lies somewhere other than its wiring, for example in its chemistry, its timing, or the signals it receives.

2 Preliminaries

2.1 Games and equilibria

A two-player game is $\Gamma = (N, (S_i)_{i\in N}, (u_i)_{i\in N})$ with players $N = \{1, 2\}$, action sets S_i and payoffs $u_i : S_1 \times S_2 \to \mathbb{R}$. It is symmetric if $S_1 = S_2 = S$ and $u_1(s, s') = u_2(s', s)$. In that case we write $u(s, s')$ for the payoff of playing s against s'.

A mixed strategy is a probability distribution $\pi \in \Delta(S)$ over actions. Payoffs extend to mixed strategies by taking expectations, $u(\pi, \pi') = \mathbb{E}_{s\sim\pi,\, s'\sim\pi'}[u(s, s')]$. The support of π is $\mathrm{supp}(\pi) = \{s : \pi(s) > 0\}$.

Definition 1 (Symmetric Nash equilibrium)**.** *π^* is a symmetric Nash equilibrium if $u(\pi^*, \pi^*) \geq u(\pi, \pi^*)$ for every $\pi \in \Delta(S)$. Equivalently, every action in $\mathrm{supp}(\pi^*)$ earns the same payoff against π^*, and no action outside the support earns more.*

Definition 2 (Evolutionarily stable strategy, ESS)**.** *π^* is an ESS if for every $\pi \neq \pi^*$, either $u(\pi^*, \pi^*) > u(\pi, \pi^*)$, or $u(\pi^*, \pi^*) = u(\pi, \pi^*)$ and $u(\pi^*, \pi) > u(\pi, \pi)$ [1].*

Definition 3 (Stable game)**.** *A symmetric game with a finite action set and payoff matrix U is strictly stable if $z^\top U z < 0$ for every nonzero z with $\mathbf{1}^\top z = 0$. Such a game has exactly one symmetric equilibrium, and standard learning dynamics converge to it [11].*

2.2 Distance from equilibrium

Definition 4 (Exploitability)**.** *The exploitability (ex-ante utility loss) of a strategy π against an opponent strategy π' is*

$$\ell(\pi; \pi') = \max_{s\in S} u(s, \pi') - u(\pi, \pi').$$

It is how much a player using π could gain by switching to the best response. π^* is a symmetric equilibrium exactly when $\ell(\pi^*; \pi^*) = 0$ [12].

2.3 Policies and policy-gradient learning

A policy $\pi_\theta(s \mid x)$ is a probability distribution over actions produced by a network with parameters θ from an input (context) x. Given sampled actions s_j with payoffs u_j and a baseline $\bar{u}$ (the average payoff so far), the REINFORCE gradient estimate [14] is

$$\hat{g} = \frac{1}{B} \sum_{j=1}^{B} (u_j - \bar{u})\, \nabla_\theta \log \pi_\theta(s_j \mid x).$$

The quantity $A_j = u_j - \bar{u}$ is called the advantage. Actions that did better than usual become more likely, and actions that did worse become less likely.

Proximal policy optimisation (PPO) [15] reuses one batch of experience for several updates. It measures how much the policy has moved using the ratio $r_j(\theta) = \pi_\theta(s_j \mid x)/\pi_{\theta_{\mathrm{old}}}(s_j \mid x)$ and maximises

$$L^{\mathrm{clip}}(\theta) = \frac{1}{B} \sum_j \min\Big(r_j(\theta) A_j,\ \mathrm{clip}\,\big(r_j(\theta), 1-\epsilon, 1+\epsilon\big) A_j \Big),$$

which stops the policy from moving too far from the one that collected the data.

2.4 Connectomes as graphs

A connectome is a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, w)$. Its vertices are neurons, an edge $(j \to i)$ means neuron j makes synapses onto neuron i, and $w_{ij} \in \mathbb{N}$ is the number of those synapses. Each neuron also has a predicted neurotransmitter [16]. Under Dale's principle this fixes the sign $\sigma_j \in \{+1, -1, 0\}$ of all of neuron j's outgoing connections: $+1$ for excitatory, -1 for inhibitory, and 0 for modulatory or unknown. For a set of neurons $\mathcal{V}'$, $\mathcal{G}[\mathcal{V}']$ is the subgraph containing only those neurons and the edges between them. The in-degree d_i^{in} and out-degree d_i^{out} count the incoming and outgoing edges of neuron i.

A degree-preserving rewire [13] repeatedly picks two edges $(a \to b)$ and $(c \to d)$ and swaps them to $(a \to d)$ and $(c \to b)$, skipping swaps that would create self-loops or duplicate edges. Every neuron keeps its in- and out-degree, and everything else about the wiring is randomised.

3 Problem Formulation

3.1 The problem

Two agents repeatedly play the war of attrition. In each round, each agent picks how long to stay in the fight. Each agent's strategy comes from a neural network, and the network is wired exactly like the fly's give-up circuit: neurons from the connectome become units, and synapses become connections. The agents improve their strategies over many rounds using reinforcement learning, only from the payoffs they experience. We know exactly what the equilibrium strategy is, so we can track how close the agents get to it and how fast. We then repeat everything with networks of the same size whose wiring is not the fly's. The problem is to find out whether the real wiring makes any difference to how well and how fast the agents learn the equilibrium.

3.2 The game

Let $V > 0$ be the value of the prize and $c > 0$ the cost of fighting per unit time. Each player picks a quitting time $t \geq 0$. Both pay $c \min(t, t')$, and the one who stays longer wins V:

$$u(t, t') = \begin{cases} V - c\,t' & t > t', \\ -c\,t & t < t', \\ V/2 - c\,t & t = t'. \end{cases}$$

Proposition 1 (1, 2)**.** *The game has a unique symmetric equilibrium, which is also its ESS:* $T \sim \text{Exp}(c/V)$*, so* $P(T > t) = e^{-ct/V}$ *and* $\mathbb{E}[T] = V/c$*. The equilibrium payoff is* 0*.*

We use $V = c = 1$, so the equilibrium is $T \sim \text{Exp}(1)$. Time is placed on a grid $\mathcal{T} = \{t_k = k\Delta\}_{k=0}^{n-1}$ with $n = 64$ points, $T_{\max} = 8$ and $\Delta = T_{\max}/(n-1)$. On the grid the game becomes a 64×64 matrix game $U = V\,W + C$, with $W_{kl} = \mathbb{1}[k > l] + \frac{1}{2}\mathbb{1}[k = l]$ and $C_{kl} = -c \min(t_k, t_l)$. This grid game is strictly stable, so it has one symmetric equilibrium p^*. It is geometric, $p_k^* \propto r^k$ with $r = \sqrt{1 + (c\Delta/V)^2} - c\Delta/V$, and matches the exponential rounded down onto the grid.

3.3 The circuit

Let $\mathcal{G}^\star = (\mathcal{V}^\star, \mathcal{E}^\star, w, \sigma)$ be the MaleCNS v1.0 connectome [6], keeping only edges with $w_{ij} \geq 5$ synapses. The three core cell populations from Ventimiglia et al. [3] are

$$P = \{\text{PPL101–PPL108}\}\ (16\ \text{neurons}), \qquad M = \{\text{MBON18, MBON19}\}\ (6), \qquad Q = \{\text{AVLP727m}\}\ (5),$$

where AVLP727m is the MaleCNS name for the Tk-GAL4^{FruM} neurons. Let X be every neuron on a directed path of at most 3 synapses from M to Q, and let Y be every neuron that exchanges at least 400 synapses with $P \cup M \cup Q \cup X$. The circuit is

$$\mathcal{G} = \mathcal{G}^\star[P \cup M \cup Q \cup X \cup Y], \qquad |\mathcal{V}| = 258,\ |\mathcal{E}| = 4{,}952.$$

The input neurons are $\mathcal{I} = P$ and the output neurons are $\mathcal{O} = Q$.

3.4 Agents and controls

For any graph $\mathcal{G}'$ on the same neurons, let $\Pi(\mathcal{G}')$ be the set of policies $\pi_\theta(\cdot \mid x) \in \Delta(\mathcal{T})$ computed by a recurrent network whose connections are exactly the edges of $\mathcal{G}'$ (Section 4 gives the exact form). We compare the real circuit $\mathcal{G}$ against:

- $\mathcal{D}_{\mathrm{rew}}(\mathcal{G})$, random degree-preserving rewires of $\mathcal{G}$;
- $\mathcal{D}_{\mathrm{rand}}(\mathcal{G})$, random graphs on $\mathcal{V}$ with the same number of edges;
- a dense network Π_{MLP} (two hidden layers) with about the same number of trainable parameters.

All circuit-based networks keep the same neurons, the same input and output neurons, and the same neuron signs, so they have exactly the same number of parameters.

3.5 The research question

Let θ_τ be an agent's parameters after τ rounds of learning, and ℓ_τ its exploitability. Let $\tau_\varepsilon = \min\{\tau : \ell_\tau \le \varepsilon\}$ be the first round at which exploitability falls below ε. For a learning rule $\mathcal{L}$ and training length T, we want to estimate

$$\delta_{\mathrm{rew}} = \mathbb{E}\big[\ell_T(\Pi(\mathcal{G}))\big] - \mathbb{E}_{\mathcal{G}' \sim \mathcal{D}_{\mathrm{rew}}(\mathcal{G})}\big[\ell_T(\Pi(\mathcal{G}'))\big], \qquad \delta_{\mathrm{rand}} = \mathbb{E}\big[\ell_T(\Pi(\mathcal{G}))\big] - \mathbb{E}_{\mathcal{G}' \sim \mathcal{D}_{\mathrm{rand}}(\mathcal{G})}\big[\ell_T(\Pi(\mathcal{G}'))\big],$$

and the same differences for τ_ε, and test whether they are zero. δ_{rew} measures the effect of the fly's specific wiring. $\delta_{\mathrm{rand}} - \delta_{\mathrm{rew}}$ measures the effect of the degree pattern. Comparing $\Pi(\mathcal{G}')$ with Π_{MLP} measures the effect of using a sparse recurrent circuit at all.

4 Methodology: How the Fly Circuit Learns

This section explains, step by step, how a piece of the fly connectome becomes an agent that plays the game and improves through reinforcement learning. Figure 1 shows the whole loop and Algorithm 1 gives it as pseudocode.

4.1 Step 1: from connectome to network

Every one of the 258 neurons in $\mathcal{G}$ becomes one unit in a recurrent network, with an activity value h_i. Every one of the 4,952 edges becomes one trainable weight θ_{ij}. Put these in a matrix A_θ with $(A_\theta)_{ij} = \theta_{ij}$ if $(j \to i) \in \mathcal{E}$, and 0 otherwise. The zeros are fixed: the network can never create a connection that the fly does not have. At the start of training the weights are random, with scale $1/\sqrt{d_i^{\mathrm{in}}}$.

In the sign-constrained version we also respect Dale's principle. For a neuron with $\sigma_j \neq 0$ we write $\theta_{ij} = \sigma_j \,\mathrm{softplus}(\tilde{\theta}_{ij})$ and train $\tilde{\theta}_{ij}$ instead, so an excitatory neuron's outgoing weights stay positive and an inhibitory neuron's stay negative throughout learning.

4.2 Step 2: one forward pass (the fly "thinks")

The game context $x = V/c$ enters the circuit only through the 16 PPL1 input neurons. The signal then spreads through the wiring for $L = 6$ steps:

$$h^{(0)} = 0, \qquad h^{(\ell+1)} = \tanh\big(A_\theta h^{(\ell)} + E_{\mathcal{I}} W_{\mathrm{in}} x + b\big), \quad \ell = 0, \dots, L-1.$$

Here $E_{\mathcal{I}}$ places the input on the PPL1 neurons, $W_{\mathrm{in}} \in \mathbb{R}^{16}$ is the input weight of each PPL1 neuron, and $b \in \mathbb{R}^{258}$ is a bias per neuron. At each step a neuron adds up the activity of the neurons that synapse onto it, weighted by θ_{ij}, and passes it through $\tanh$. We use $L = 6$ because the core path PPL1 $\to$ MBON $\to$ two connecting neurons $\to$ AVLP727m is 4 synapses long, and 6 steps leave some slack.

After L steps we read the activity of the 5 AVLP727m output neurons and turn it into a probability for each of the 64 quitting times:

$$z = R^\top h_{\mathcal{O}}^{(L)} + c_0 \in \mathbb{R}^{64}, \qquad \pi_\theta(t_k \mid x) = \frac{e^{z_k}}{\sum_m e^{z_m}}.$$

In total the network has 4,952 edge weights, 16 input weights, 258 biases, and $5 \times 64 + 64$ readout weights, which is 5,610 trainable parameters.

4.3 Step 3: playing a round

Each agent samples a quitting time $t \sim \pi_\theta(\cdot \mid x)$, and its opponent samples t' from its own policy. The payoff $u(t, t')$ is computed from the rule above. Ties are settled with a fair coin for the prize, so each match is a clear win or loss.

4.4 Step 4: learning from the payoff (the RL update)

For a sampled quitting time t_k with advantage $A = u - \bar{u}$, REINFORCE changes the parameters in the direction $A \nabla_\theta \log \pi_\theta(t_k \mid x)$. The interesting part is how this gradient reaches the synapses. It is computed by backpropagation through the 6 unrolled steps, and it travels backwards along the same wiring the signal travelled forwards:

$$\begin{aligned}
\frac{\partial \log \pi_\theta(t_k \mid x)}{\partial z} &= e_k - \pi_\theta(\cdot \mid x), \\
\delta^{(L)} &= \big(1 - (h^{(L)})^2\big) \odot E_{\mathcal{O}} R \,\big(e_k - \pi_\theta(\cdot \mid x)\big), \\
\delta^{(\ell)} &= \big(1 - (h^{(\ell)})^2\big) \odot A_\theta^\top \delta^{(\ell+1)}, \qquad \ell = L-1, \dots, 1, \\
\frac{\partial \log \pi_\theta(t_k \mid x)}{\partial \theta_{ij}} &= \sum_{\ell=0}^{L-1} \delta_i^{(\ell+1)} \, h_j^{(\ell)}, \qquad (j \to i) \in \mathcal{E}.
\end{aligned}$$

Here e_k is the one-hot vector for t_k and $E_{\mathcal{O}}$ places the readout on the AVLP727m neurons. In words: the error starts at the AVLP727m neurons and flows backwards through $A_\theta^\top$, which has the same nonzero pattern as the fly wiring. A synapse $j \to i$ changes in proportion to how active j was and how much blame reaches i. So credit assignment itself is shaped by the connectome, and this is where a difference between real and shuffled wiring could show up.

PPO version. Instead of one REINFORCE step per round, PPO takes 4 gradient steps on L^{clip} with $\epsilon = 0.2$, using the same batch of matches and the same backward pass through the circuit.

Loss-weighted version. Real flies react more strongly to losing than to winning. To model this we weight each sample by α_{loss} after a loss and α_{win} after a win, with ratio $\rho = \alpha_{\text{loss}}/\alpha_{\text{win}}$ and $\alpha_{\text{win}} = \frac{2}{1+\rho}$, $\alpha_{\text{loss}} = \frac{2\rho}{1+\rho}$ so the average step size stays the same. For $\rho > 1$ the learner no longer settles on the true equilibrium but on a shifted one with a decreasing quitting rate,

$$h(t) = \frac{\rho c}{V + (\rho - 1)(ct + \bar{u})}, \qquad \bar{u} = \frac{V(\rho - 1)}{4\rho},$$

which gives us an exact target to check against.

4.5 Step 5: who plays whom

We train in two modes.

- Exact self-play. One shared policy plays against a frozen copy of itself, and we compute the exact expected payoff over the 64-point grid instead of sampling. The update is $\theta \leftarrow \theta + \eta \nabla_\theta u(\pi_\theta, \text{sg}(\pi_\theta))$. There is no sampling noise, so this tells us what each network can learn in the best case.
- Sampled self-play (the RL setting). A population of $M = 8$ separate agents, each with its own copy of the circuit, is randomly re-paired every round. Each agent plays $B = 128$ matches per round and learns only from its own quitting times and payoffs, using REINFORCE or PPO. Random re-pairing matters: two fixed opponents can drift into a lopsided equilibrium where one always quits [17], while a mixed population keeps the exponential as the stable outcome.

All networks use the same optimiser, Adam [18] with no momentum ($\beta_1 = 0$) and a cosine learning-rate decay. With momentum, gradient play tends to circle around mixed equilibria instead of settling.

4.6 A second way to use the circuit: frozen synapses

Following Jin et al. [8], we also build a version where the synapse weights are not trained at all. They are fixed to the real signed synapse counts, $W_{ij} = \sigma_j w_{ij}$, scaled so the largest eigenvalue has size 1. Each neuron instead gets a small learned descriptor $\eta_i \in \mathbb{R}^9$, and a shared update network f_ψ decides how each neuron responds to its incoming messages:

$$m^{(\ell)} = W h^{(\ell)}, \qquad h_i^{(\ell+1)} = f_\psi\big(h_i^{(\ell)}, m_i^{(\ell)}, \eta_i\big).$$

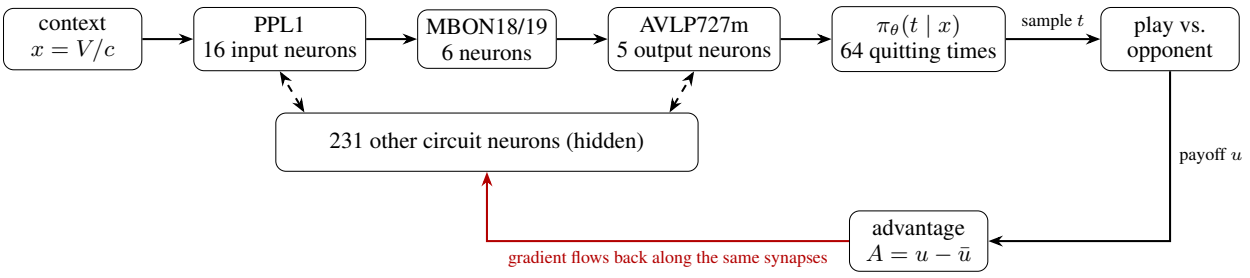


Figure 1: One learning step. The context enters through the PPL1 neurons, spreads through the 258-neuron circuit for 6 steps, and is read out from the AVLP727m neurons as a distribution over quitting times. The agent samples a time, plays, and receives a payoff. The advantage is sent back through the circuit by backpropagation, which follows the fly's wiring in reverse and updates each synapse weight.

Algorithm 1 Sampled self-play with circuit-wired agents

Build $\mathcal{G}$ from the connectome; for each of M agents, create weights θ^m on the edges of $\mathcal{G}$
for round $\tau = 1, \dots, T$ **do**
  Randomly pair the agents
  for each agent m **do**
    Run the circuit for L steps on x to get $\pi_{\theta^m}(\cdot \mid x)$ ▷ Step 2
    Play B matches: sample $t_j \sim \pi_{\theta^m}$, get payoff u_j and outcome o_j ▷ Step 3
    $A_j \leftarrow \alpha(o_j)\,(u_j - \bar{u}^m)$; $\bar{u}^m \leftarrow (1-\beta)\,\bar{u}^m + \beta \cdot \mathrm{mean}_j(u_j)$
    Backpropagate through the unrolled circuit to get $\hat{g}^m = \frac{1}{B}\sum_j A_j \nabla \log \pi_{\theta^m}(t_j \mid x)$ ▷ Step 4
    Update θ^m with Adam (or take 4 PPO steps on L^{clip})
  end for
  Every 25 rounds: compute exact exploitability of each agent against the others
end for

Only η, ψ, the input encoder and gate, and the readout are learned. This asks a stricter question: can the fly's actual synapse strengths, left untouched, carry the learning?

4.7 A note on biological realism

Backpropagation is not how a real fly learns. We use it because it is the standard way to train a network, and our question is about the wiring as a learning substrate, not about the fly's own learning rule. In the fly, PPL1 dopamine is thought to act as a teaching signal. We feed the context into PPL1 as an input, which is a modelling choice we keep fixed across all networks so the comparison stays fair.

5 Proposed Solutions

5.1 What we will compare

We will train the following agents, all with close to the same number of trainable parameters (within 5%):

- an oracle that plays the known grid equilibrium, used as a reference;
- a tabular policy (one learnable score per quitting time), as a sanity check;
- a dense two-layer network;
- the real circuit, a degree-preserving rewire and a random sparse graph, each with and without sign constraints;
- the frozen-synapse version of the real circuit and of its two controls.

Each is trained with exact self-play and with sampled self-play (REINFORCE and PPO), and in sampled mode with loss weighting $\rho \in \{1, 2, 4, 8\}$.

5.2 Metrics

- Exploitability ℓ, computed exactly on the grid with no sampling. In sampled mode, each agent is measured against the mix of the other agents. This is the main metric.
- Time to threshold τ_ε: the first round at which $\ell \le \varepsilon$, with runs that never get there marked as censored.
- Distance of the learned quitting-time distribution from the equilibrium, using total variation and the Wasserstein-1 distance.
- The learned survival curve $P(T > t)$, compared with e^{-t} (and with the shifted equilibrium when $\rho > 1$).
- Stability: how much the policy still moves in the last 10% of training.
- A loser effect: how much an agent's next quitting time drops after a round with many losses, which is what real defeated flies show.

5.3 Results we plan to show

- Learning curves of exploitability over training for every agent, in both modes.
- A table of final exploitability and time to threshold for every agent, with confidence intervals.
- The main comparison: real circuit versus rewire versus random graph, with the differences δ_{rew} and δ_{rand} and whether they are significant.
- The learned survival curves against the exponential equilibrium.
- The effect of sign constraints, and of trained versus frozen synapses.
- REINFORCE versus PPO, to see whether a more stable learning rule changes the picture.
- Final exploitability as a function of ρ, compared with the predicted shifted equilibrium, and whether the wiring changes the loser effect.

5.4 How we will do it

We will run 20 random seeds per setting in exact mode and 10 in sampled mode. Learning rates will be tuned per agent on separate seeds, and all circuit-based agents will also be run at one shared learning rate, so that the only difference between them is the wiring. For every comparison we will report the mean with a 95% bootstrap confidence interval, an effect size, and a permutation test with Holm correction for multiple comparisons [19]. We will call a difference real only if its interval excludes zero and the corrected p-value is below 0.05. Every circuit will be rebuilt from the raw MaleCNS files and checked against a stored checksum, so the results are reproducible.

As future scope, the same setup extends to a version of the game where each fly privately knows how much it values the prize, which forces the circuit to turn its input into a value-dependent quitting time.

6 Contribution

Member	Tasks
Suhani	Problem design; extracting the give-up circuit from the MaleCNS connectome and building the rewired and random controls; designing the circuit-wired and frozen-synapse policy networks; the training, evaluation and statistics pipeline; running the experiments.
Mayank Choudhary	Mathematical analysis of the game: the exponential equilibrium, the grid equilibrium and its stability, and the shifted equilibrium under loss-weighted learning; the reinforcement learning formulation (exact self-play, REINFORCE, PPO) and the exploitability metric.
Krina Paghadar	Wrote the Preliminaries section; worked with Mayank on the mathematical formulation of the problem.
Nakshatra Lad	Literature survey on the war of attrition and connectome-based learning; drafting the Introduction; figures and report formatting.

References

[1] John Maynard Smith. The theory of games and the evolution of animal conflicts. *Journal of Theoretical Biology*, 47(1):209–221, 1974.

[2] D. T. Bishop and C. Cannings. A generalized war of attrition. *Journal of Theoretical Biology*, 70(1):85–124, 1978.

[3] D. Ventimiglia et al. Neural circuits regulating social dominance implement a strategy predicted by evolutionary game theory. *bioRxiv*, 2026. doi:10.64898/2026.02.20.706896.

[4] Louis K. Scheffer et al. A connectome and analysis of the adult *Drosophila* central brain. *eLife*, 9:e57443, 2020.

[5] Sven Dorkenwald et al. Neuronal wiring diagram of an adult brain. *Nature*, 634:124–138, 2024.

[6] S. Berg et al. Sexual dimorphism in the complete *Drosophila* male central nervous system connectome. *Cell*, 189, 2026.

[7] Janne K. Lappalainen et al. Connectome-constrained networks predict neural activity across the fly visual system. *Nature*, 634:1132–1140, 2024.

[8] Zehao Jin, Yaoye Zhu, Chen Zhang, and Yanan Sui. Whole-brain connectomic graph model enables whole-body locomotion control in fruit fly. *arXiv preprint arXiv:2602.17997*, 2026.

[9] Roman Vaxenburg et al. Whole-body physics simulation of fruit fly locomotion. *Nature*, 2025.

[10] Drew Fudenberg and David K. Levine. *The Theory of Learning in Games*. MIT Press, 1998.

[11] Josef Hofbauer and William H. Sandholm. Stable games and their dynamics. *Journal of Economic Theory*, 144 (4):1665–1693, 2009.

[12] Martin Bichler, Maximilian Fichtl, Stefan Heidekrüger, Nils Kohring, and Paul Sutterer. Learning equilibria in symmetric auction games using artificial neural networks. *Nature Machine Intelligence*, 3:687–695, 2021.

[13] Sergei Maslov and Kim Sneppen. Specificity and stability in topology of protein networks. *Science*, 296(5569): 910–913, 2002.

[14] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

[15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[16] Nils Eckstein et al. Neurotransmitter classification from electron microscopy images at synaptic sites in *Drosophila melanogaster*. *Cell*, 187:2574–2594, 2024.

[17] Reinhard Selten. A note on evolutionarily stable strategies in asymmetric animal conflicts. *Journal of Theoretical Biology*, 84(1):93–101, 1980.

[18] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[19] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2): 65–70, 1979.